

SQL QUERY LINTER & STYLE FIXER

Project Technical Documentation & System Specifications

1. Problem Statement

In modern data-driven architectures, database development teams frequently inherit legacy software systems containing poorly structured, unformatted, and inefficient SQL queries. Standard queries often suffer from several critical issues:

- **Naming Discrepancies:** Mix of camelCase, PascalCase, and lowercase which leads to inconsistencies in query audits.
- **Legacy Syntax:** Extensive usage of implicit comma-joins (e.g., FROM table_a, table_b WHERE table_a.id = table_b.id) which obscures join relationships and makes queries hard to maintain.
- **Query Complexity:** Deeply nested subqueries that degrade readability and hinder performance optimization.
- **Anti-patterns:** Unrestricted use of SELECT * which drives unnecessary cloud database billing costs.

Manual refactoring is labor-intensive and highly error-prone. Standard linters only fix trivial whitespace or capitalization, whereas direct LLM optimizations can hallucinate and generate syntactically broken SQL code.

2. Technologies Used

The application leverages a hybrid architectural stack consisting of deterministic client-side parser rules combined with LLM-based refactoring agents and Abstract Syntax Tree (AST) validation critics:

- **Python:** Core language for regex engine scripting and network routing.
- **Streamlit (v1.x):** Modern dashboard framework used to design the visual UI, sandbox workspace, and cloud file uploading tabs.
- **Google GenAI SDK & gemini-2.5-flash:** Advanced LLM used to analyze query semantics, restructure subqueries into CTEs, and translate implicit joins.
- **sqlglot:** Multi-dialect SQL parser used locally to validate LLM query syntax and provide a zero-cost local AST pretty-printer.
- **difflib:** Native Python library utilized to compute structured unified diff comparison displays.

3. System Architecture

The system follows a sequential pipeline where data flows through three distinct processing filters to guarantee clean formatting and valid execution:

Pipeline Stage	Components Involved	Operation Performed
Input Sanitizer	Regex rules	Strips markdown backticks, asterisks (*), and underscores () wrapping words.

Phase 1: Linter	Deterministic parser	Capitalizes SQL keywords, normalizes casing to snake_case, logs SELECT * warnings.
Phase 2: Refactor	Google Gemini API / sqlglot AST	Converts comma-joins to INNER JOINs, extracts subqueries into CTEs.
Verification Loop	sqlglot Validator	Validates output syntax locally. Feeds errors back to LLM to self-correct up to 3 times.

4. Modules Overview

The source codebase is divided into modular, decoupled functional files:

4.1. Local Linter & Sanitizer (`lint_engine.py`)

Houses `strip_markdown_artifacts()` which uses negative lookbehinds/lookaheads to prevent mangling database column names like `employee_shift_start`. Executes `lint_sql()` to enforce keyword capitalization and casing conversion.

4.2. Refactoring Engine & loop (`llm_agent.py`)

Handles GenAI integration, configures the temperature (0.1), few-shot learning instruction templates, and executes the self-correcting logic loop. Automatically falls back to rule-based local AST formatting using `sqlglot` if no Google Gemini API Key is configured.

4.3. Interface Orchestrator (`app.py`)

Defines the dual-mode launcher. Runs either CLI folder-batch operations or starts the web browser dashboard configured with custom CSS styling, markdown diff comparisons, and file upload structures.

5. Workflow Specification

When a user optimizes a query (via files, playground, or CLI):

1. **Markdown Cleanup**: Text is sanitized to remove bold/italic elements.
2. **Style Formatting (Phase 1)**: Keywords are capitalized and camelCase columns are transformed into snake_case. Warnings are raised if `SELECT *` is found.
3. **GenAI Refactoring (Phase 2)**: Standardized SQL is routed to gemini-2.5-flash with few-shot rules for explicit joins and CTEs.
4. **Syntax Correction**: `sqlglot` tries to parse the generated SQL. If it fails, the syntax exception error is appended to a correction prompt and resent.
5. **Display & Download**: The final verified query is output inside a copyable code block, and a colored diff is rendered.

6. Project Advantages

- **Hybrid Reliability**: Capitalization and casing are handled locally at zero cost, reserving expensive LLM tokens purely for semantic structure.
- **Self-Healing Code**: The agent loop prevents typical AI errors, ensuring that the user never receives broken SQL statements.
- **Cloud Deployability**: In-memory file uploader enables hosting on isolated platforms (like Streamlit Community Cloud) without directory permission barriers.
- **Developer Friendly**: Dual-mode CLI and UI support makes it adaptable for single-query debugging or developer pipeline automation.

7. Conclusion

The SQL Query Linter & Style Fixer delivers an enterprise-grade, lightweight solution that converts legacy, unoptimized database queries into highly readable, structured, and modern SQL. By combining deterministic tokenizers with self-correcting GenAI pipelines, it achieves a balance of validation security and semantic reasoning that typical rule-based formatting tools cannot match.